Methodology for the Design and Evaluation of CampusOS, a Campus Venue and Event Platform

Gaurav Jadhav, Atharva Desai, Aditya Patil, Varad Rajboinwad, Tushar Deshpande, Chaitali Shahapurkar, Tanishka Patil, Srushti Mane, Sarvesh Khaladkar, Shravani Khandzode, Omkar Shinde, Gayatri Muttepawar

Department of Information Technology
Marathwada Mitra Mandal's College of Engineering
Pune, India

I. Methodology

A. Research Approach

This work follows a design-and-evaluate approach. We build a working system against a fixed set of requirements, then test whether it keeps its most important rule under conditions chosen to break it. That rule is that no two approved bookings may overlap on one venue. Fig. 1 shows the stages, and the sections below describe each one. Results are left out on purpose, since this document covers only the method.

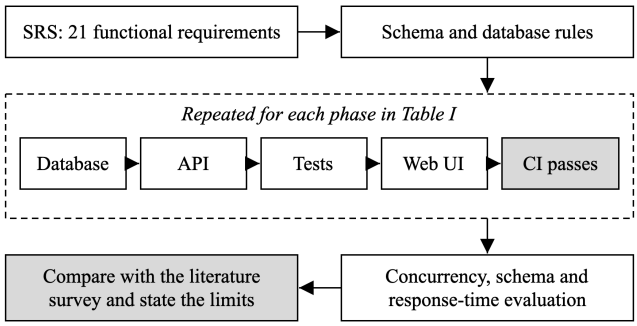


Fig. 1. Stages of the method. Each phase runs the inner loop and is complete only when continuous integration (CI) passes.

B. Requirements

Requirements come from the Software Requirements Specification (SRS) of the project, which lists 21 functional requirements, FR1 to FR21, along with targets such as a response time under 3 s and support for 500 simultaneous users. The starting point was the paper venue request used at our college. We did not carry out user interviews, personas, wireframe reviews or usability tests, so the requirements rest on the SRS and on our own knowledge of the process as students.

C. Literature Survey

The survey is a narrative one. Application papers were searched through OpenAlex and Crossref with strings such as "venue booking system university conflict" and "double booking concurrency reservation system database locking," and papers from 2016 onward that described a working system were kept. Each paper was compared on approval step, conflict handling and reported concurrency evidence. Older database and security papers were added for theory. Every DOI was checked on Crossref. This search was run after the system was built, so it places the design against published work and did not produce the design. Three application papers were read in full and seven by abstract only.

D. Incremental Development

Work was split into the phases in Table I, each a vertical slice of features. Within a phase the order was fixed: database first, then the API with manual checks in Postman, then automated tests and the web screens. The frontend was never built against mocked endpoints first. A phase counted as complete only when every test passed in CI on GitHub. Later phases then applied second-year course concepts, such as data structures and operating system models, in places where a feature had a use for them.

TABLE I
Development Phases

Phase	Scope	Requirements
0	Schema, seed data, API skeleton, test setup	Foundation
1	Sign-in and roles	FR1 to FR5
2	Venues and scheduling engine	FR6 to FR10
3	Approval workflow and clubs	FR11 to FR13
4	Events and seat reservation	FR14 to FR17
5	Notifications and reminders	FR19
6	Dashboards, audit trail, reports	FR18, FR20, FR21

E. System Design

The system has three layers: a React web client, an Express API in the REST style [1], and a PostgreSQL 16 database with 24 tables, normalised apart from a few documented exceptions. Access follows role-based control [2], [3] with five roles. Sign-in uses a short-lived JSON Web Token [4] with a rotating refresh cookie, and passwords are stored with bcrypt [5]. Rules that must never be broken are written twice, in the application code and as database constraints and triggers, so a fault in one layer is caught by the other.

F. Booking Method

A check followed by a separate write allows two requests to pass together, which is the write skew described by Berenson *et al.* [6]. The conflicting row does not exist yet, so it cannot be locked directly, which is the phantom problem of

Eswaran *et al.* [7]. We therefore lock a row that always exists, the venue. Every operation that can occupy a slot runs these steps in one transaction:

1) Take a row lock on the venue with SELECT ... FOR UPDATE [8], so writers for one venue wait in turn.
2) Read the existing bookings for that venue and test each one with (1).
3) On a clash, refuse the operation and give the reason. Otherwise write the booking.
4) On approval, also reject the competing pending requests in the same transaction, then commit.

With a buffer B for setup and cleaning, a new booking clashes with an existing one when

$$(S_n < E_e + B) \text{ AND } (E_n + B > S_e) \quad (1)$$

where S and E are start and end times and the subscripts mark the new and existing bookings. This treats venues like the machines in interval scheduling [9]. The default buffer is 15 minutes, and a venue can set its own value from 0 to 120 minutes.

As a second layer, an exclusion constraint on venue and time range [10] makes PostgreSQL refuse two overlapping approved rows, even if the application is wrong or bypassed. It uses the raw time window, so the buffer is enforced only by the application query. To prevent deadlock [11], all code takes locks in one fixed order, starting from venues and bookings and ending at clubs, club members and users, and takes rows of one table in ascending key order. This removes circular wait.

G. Supporting Methods

Seat reservation uses the same locking pattern, backed by a capacity check and a trigger in the database. A background worker sends reminders 48 hours and 2 hours before an event and claims its work with FOR UPDATE SKIP LOCKED [8]. The audit table is append-only through a trigger. Event recommendations use a simple weighted score from a student's past registrations, clubs and department, with no machine learning, because a single college has too little history to train a model.

H. Test Method

Table II lists the levels of testing. The integration, concurrency and schema levels run against a real PostgreSQL database and not a mock, because locks and constraints cannot be checked any other way. For a formal run, the database is dropped and rebuilt from the schema and seed files, and suites run one at a time because they share it. The run recorded in our test report used an Apple M4 laptop with 16 GB of memory, Node.js 24 and PostgreSQL 16.

CI runs on every pull request and every push to the main branch. It runs the backend tests against a PostgreSQL service with a coverage gate, ESLint, frontend tests, a production build and a dependency audit. The backend gate is currently set at 97% of statements, 89% of branches, 97% of functions and 98% of lines. The gates were raised over time and not lowered.

TABLE II
LEVELS OF TESTING

Level	What it checks	Tools
Unit	One function or class	Jest, Vitest
Integration	HTTP request through routes, services and SQL	Jest, supertest, PostgreSQL
Concurrency	Many requests at once on one row	Jest, supertest, PostgreSQL
Schema	Constraints and triggers with the API bypassed	Jest, PostgreSQL
Flow	Screens driven as a user would, API mocked at the network edge	Vitest, Testing Library, MSW
Equivalence	New code against a reference on seeded random inputs	Jest

I. Concurrency Evaluation

Table III gives the scenarios. Each one fires its calls at the same moment, with a connection pool large enough that they really overlap. A scenario passes only if its exact condition holds, since a rate such as 99% would still allow a double booking.

TABLE III
CONCURRENCY AND BYPASS SCENARIOS

Scenario	Load	Pass condition
Slot approval	20 approvals of competing requests for one slot	Exactly 1 succeeds and 19 are refused
Last seat	20 students reserve the last seat	Exactly 1 gets it; seats never exceed capacity
Publish	20 publishes of one event	Exactly 1 succeeds, with one broadcast
Lock order	15 addMember and 15 changeRole calls on one club	No deadlock error (40P01); all end as members
Overlap bypass	Overlapping approved row inserted in SQL	Rejected by the exclusion constraint
Capacity bypass	Registration past capacity in SQL	Rejected by the trigger
Audit bypass	Update or delete of an audit row	Rejected by the trigger

To show that a test can really detect its fault, the lock-order and publish tests were confirmed to fail against the old code before its fix. These two faults came from the project itself. The deadlock was first found by reading the code and could also be reproduced by hand in two database sessions.

J. Response-Time Probe

A script signs in as a demo user and times GET requests on five endpoints, sending 500 requests per endpoint with 25 and then 100 in flight, and records the median, 95th percentile

and maximum. The values are compared with the SRS targets of 3 s in general and 2 s for the dashboard. This is a probe and not a load test, as it uses one process on one laptop with a small seeded database and no network delay.

K. Limits of the Method

The method has known limits. The system has not been deployed, and no real users have taken part, so usefulness to a college is not evaluated. The 500-user target and the availability target are not tested. Real email delivery is replaced by a mock, flow tests run in a simulated browser and not on real devices, and no cross-browser or accessibility testing has been done. We also wrote the tests for our own system, so the evaluation is not independent. The survey did not search IEEE Xplore or Scopus directly and relies on abstracts for seven papers.

REFERENCES

[1] R. T. Fielding and R. N. Taylor, “Principled design of the modern Web architecture,” *ACM Trans. Internet Technol.*, vol. 2, no. 2, pp. 115–150, May 2002, doi: 10.1145/514183.514185.

[2] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996, doi: 10.1109/2.485845.

[3] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: 10.1145/501978.501980.

[4] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[5] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annu. Tech. Conf.*, 1999, pp. 81–91.

[6] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O’Neil, and P. O’Neil, “A critique of ANSI SQL isolation levels,” in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1995, pp. 1–10, doi: 10.1145/223784.223785.

[7] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, “The notions of consistency and predicate locks in a database system,” *Commun. ACM*, vol. 19, no. 11, pp. 624–633, Nov. 1976, doi: 10.1145/360363.360369.

[8] PostgreSQL Global Development Group. “Explicit locking,” in *PostgreSQL 16 Documentation*. Accessed: Nov. 2, 2026. [Online]. Available: https://www.postgresql.org/docs/16/explicit-locking.html

[9] A. W. J. Kolen, J. K. Lenstra, C. H. Papadimitriou, and F. C. R. Spieksma, “Interval scheduling: A survey,” *Nav. Res. Logist.*, vol. 54, no. 5, pp. 530–543, 2007, doi: 10.1002/nav.20231.

[10] PostgreSQL Global Development Group. “Range types,” in *PostgreSQL 16 Documentation*. Accessed: Nov. 2, 2026. [Online]. Available: https://www.postgresql.org/docs/16/rangetypes.html

[11] E. G. Coffman, M. Elphick, and A. Shoshani, “System deadlocks,” *ACM Comput. Surv.*, vol. 3, no. 2, pp. 67–78, Jun. 1971, doi: 10.1145/356586.356588.